

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнила:
Студент 4 курса
Группы ИИ-26
Петручик Д.А.
Проверила:
Андренко К.В.

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Общее задание

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Выборка	Оптимизатор	Предобученная архитектура
10	STL-10 (размеченная часть)	Adam	SqueezeNet 1.1

Описание выполненной работы

Для решения задачи классификации цветных изображений STL-10 использована архитектура SqueezeNet 1.1, предобученная на ImageNet, с применением переноса обучения (transfer learning).

Адаптация модели:

- Выходной свёрточный слой классификатора с ядром 1×1 заменён на слой с 10 выходными каналами — по количеству классов STL-10.
- Исходные RGB-изображения размером 96×96 масштабированы и кадрированы до 224×224 . Сохранены три цветовых канала.
- Применена нормализация с параметрами ImageNet: $\text{mean}=[0.485, 0.456, 0.406]$, $\text{std}=[0.229, 0.224, 0.225]$.
- При обучении использованы случайное кадрирование и горизонтальное отражение изображений. При тестировании применяется центральное кадрирование.
- Выполняется дообучение всех слоёв сети.

Параметры обучения:

- Оптимизатор: Adam, как и в ЛР №1. Для дообучения SqueezeNet установлена скорость обучения $\text{lr}=0.0001$; в ЛР №1 использовалось значение $\text{lr}=0.001$.
- Функция потерь: CrossEntropyLoss.
- Количество эпох: 10.
- Размер батча: 64.
- Обучающая выборка: 5000 размеченных изображений.
- Тестовая выборка: 8000 изображений.
- Неразмеченная часть STL-10 в обучении не используется.

Результаты обучения:

Результаты на полной тестовой выборке STL-10:

SqueezeNet 1.1: accuracy=87.00%, loss=0.3758, параметров=727,626

CNN ЛР1: accuracy=53.85%, loss=2.3431, параметров=2,365,802

Код программы:

```
import argparse
import csv
import errno
import json
import math
import random
import sys
import time

from pathlib import Path
from urllib.error import URLError


# Нормализация пути при установке Python в корень диска.
if sys.platform == "win32":
    for prefix_name in ("prefix", "exec_prefix", "base_prefix", "base_exec_prefix"):
        prefix_value = getattr(sys, prefix_name)
        if len(prefix_value) == 2 and prefix_value[0].isalpha() and prefix_value[1] == ":\":
            setattr(sys, prefix_name, prefix_value + "\\")


import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from PIL import Image, ImageOps
from torch.utils.data import DataLoader, Dataset
from torchvision import datasets, models, transforms


SEED = 42
LR1_EPOCHS = 10
LR1_BATCH_SIZE = 64
LR1_LR = 0.001
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)
CLASSES = ["airplane", "bird", "car", "cat", "deer", "dog", "horse", "monkey", "ship", "truck"]
CLASSES_RU = ["самолёт", "птица", "автомобиль", "кошка", "олень", "собака",
               "лошадь", "обезьяна", "корабль", "грузовик"]
SQ_PREPROCESSING = "SqueezeNet1_1_Weights.IMAGENET1K_V1"
CNN_PREPROCESSING = "RGB96_normalize_0.5"


SOTA_REFERENCES = [
    {
        "method": "FixMatch (CTA)", "year": 2020,
        "accuracy_percent": 100.0 - 5.17, "std_pp": 0.63,
        "paper": "FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence",
        "table": "2", "labeled_train": 1000,
        "protocol": "WRN-37-2; 100000 неразмеченных изображений; 5 разбиений; CTAugment",
        "note": "Обучение с частичной разметкой.",
        "url": "https://nicholas.carlini.com/papers/2020_neurips_fixmatch.pdf",
    }
]
```

```

    },
    {
        "method": "FreeMatch", "year": 2023,
        "accuracy_percent": 100.0 - 5.63, "std_pp": 0.15,
        "paper": "FreeMatch: Self-adaptive Thresholding for Semi-supervised Learning",
        "table": "1", "labeled_train": 1000,
        "protocol": "WRN-37-2; 100000 неразмеченных изображений; 3 seed; лучшие checkpoints",
        "note": "Протокол оценки отличается от статьи FixMatch.",
        "url": "https://arxiv.org/pdf/2205.07246",
    },
    {
        "method": "ViT-L/16 + Spinal FC + background", "year": 2023,
        "accuracy_percent": 99.71, "std_pp": 0.06,
        "paper": "Reduction of Class Activation Uncertainty with Background Information",
        "table": "III", "labeled_train": 4500,
        "protocol": "500 validation; 3001 фоновых изображений; ImageNet-21k/1k pretraining",
        "note": "Опубликованный результат, не подтверждённый рекорд: автор снимает заявление "
            "о SOTA на STL-10 из-за возможного пересечения теста с ImageNet (раздел Discussion).",
        "url": "https://arxiv.org/pdf/2305.03238",
    },
]
SOTA_CAUTION = (
    "Условия экспериментов различаются: в лабораторной используются 5000 размеченных "
    "изображений, а в статьях — дополнительные данные и другие архитектуры."
)

```

Модели

```

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 24 * 24, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = x.view(x.size(0), 32 * 24 * 24)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)

def make_squeezenet(pretrained):
    weights = models.SqueezeNet1_1_Weights.DEFAULT if pretrained else None
    model = models.squeezenet1_1(weights=weights)
    model.classifier[1] = nn.Conv2d(512, len(CLASSES), kernel_size=1)

```

```

model.num_classes = len(CLASSES)
nn.init.normal_(model.classifier[1].weight, mean=0.0, std=0.01)
nn.init.zeros_(model.classifier[1].bias)
return model

```

```

def make_transforms():
    sq_train = transforms.Compose([
        transforms.Resize(256), transforms.RandomCrop(224),
        transforms.RandomHorizontalFlip(), transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
    sq_test = models.SqueezeNet1_1_Weights.DEFAULT.transforms()
    cnn_train = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ])
    cnn_test = transforms.Compose([
        transforms.Resize((96, 96)), transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ])
    return sq_train, sq_test, cnn_train, cnn_test

```

```

class TransformedDataset(Dataset):
    def __init__(self, dataset, transform):
        self.dataset, self.transform = dataset, transform

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, index):
        image, label = self.dataset[index]
        return self.transform(image), label

```

```

def make_loader(dataset, transform, batch_size, shuffle, device):
    return DataLoader(
        TransformedDataset(dataset, transform), batch_size=batch_size,
        shuffle=shuffle, num_workers=0, pin_memory=device.type == "cuda",
        generator=torch.Generator().manual_seed(SEED),
    )

```

Обучение и оценка

```

@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    criterion = nn.CrossEntropyLoss()

```

```
loss_sum, correct, total = 0.0, 0, 0
```

```
for images, labels in loader:
```

```
    images, labels = images.to(device), labels.to(device)
```

```
    logits = model(images)
```

```
    loss_sum += criterion(logits, labels).item() * labels.size(0)
```

```
    correct += (logits.argmax(dim=1) == labels).sum().item()
```

```
    total += labels.size(0)
```

```
return loss_sum / total, 100.0 * correct / total
```

```
def train_model(model, name, train_loader, test_loader, epochs, lr, device, history_path):
```

```
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
    criterion = nn.CrossEntropyLoss()
```

```
    history, training_seconds = [], 0.0
```

```
    for epoch in range(1, epochs + 1):
```

```
        model.train()
```

```
        started = time.perf_counter()
```

```
        loss_sum, correct, total = 0.0, 0, 0
```

```
        for step, (images, labels) in enumerate(train_loader, 1):
```

```
            images, labels = images.to(device), labels.to(device)
```

```
            optimizer.zero_grad(set_to_none=True)
```

```
            logits = model(images)
```

```
            loss = criterion(logits, labels)
```

```
            loss.backward()
```

```
            optimizer.step()
```

```
            loss_sum += loss.item() * labels.size(0)
```

```
            correct += (logits.argmax(dim=1) == labels).sum().item()
```

```
            total += labels.size(0)
```

```
            if step % 20 == 0 or step == len(train_loader):
```

```
                print(f'{name}: эпоха {epoch}/{epochs}, шаг {step}/{len(train_loader)}, "
```

```
                      f"loss={loss.item():.4f}", flush=True)
```

```
        epoch_seconds = time.perf_counter() - started
```

```
        training_seconds += epoch_seconds
```

```
        test_loss, test_accuracy = evaluate(model, test_loader, device)
```

```
        history.append({
```

```
            "epoch": epoch, "train_loss": loss_sum / total,
```

```
            "train_accuracy_percent": 100.0 * correct / total,
```

```
            "test_loss": test_loss, "test_accuracy": test_accuracy,
```

```
            "training_seconds": epoch_seconds,
```

```
        })
```

```
        write_csv(history_path, history)
```

```
        print(f'Error {name}: train loss={loss_sum / total:.4f}; "
```

```
              f"test loss={test_loss:.4f}; accuracy={test_accuracy:.2f}%")
```

```
    return {"history": history, "epochs": epochs, "lr": lr,
```

```
            "batch_size": train_loader.batch_size, "optimizer": "Adam",
```

```
            "training_seconds": training_seconds, "seed": SEED,
```

```
            "device": str(device), "origin": "trained_in_this_script"}
```

```
def save_checkpoint(model, path, architecture, preprocessing, metadata):
    checkpoint = dict(metadata)
    checkpoint.update({
        "model_state_dict": {k: v.detach().cpu() for k, v in model.state_dict().items()},
        "architecture": architecture, "classes": CLASSES, "preprocessing": preprocessing,
    })
    temporary = path.with_suffix(".pth.tmp")
    torch.save(checkpoint, temporary)
    temporary.replace(path)
```

```
def load_checkpoint(model, path, architecture, preprocessing):
    checkpoint = torch.load(path, map_location="cpu", weights_only=True)
    metadata = {}
    if isinstance(checkpoint, dict) and "model_state_dict" in checkpoint:
        for key, expected in (("architecture", architecture), ("classes", CLASSES),
                               ("preprocessing", preprocessing)):
            if key in checkpoint and checkpoint[key] != expected:
                raise ValueError(f"В {path} несовместимое поле {key}: {checkpoint[key]}")
        state = checkpoint["model_state_dict"]
        metadata = {k: v for k, v in checkpoint.items() if k != "model_state_dict"}
    else:
        state = checkpoint
    try:
        model.load_state_dict(state, strict=True)
    except RuntimeError as exc:
        raise ValueError(f"Веса {path} не соответствуют архитектуре {architecture}. "
                          "Требуется архитектура CNN из ЛП №1.") from exc
    print(f"Загружены веса: {path}")
    return metadata
```

```
def write_csv(path, rows):
    if not rows:
        return
    with path.open("w", encoding="utf-8-sig", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
```

Сравнение моделей

```
def plot_results(results, histories, reference_rows, output_dir):
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    for result in results:
        name = result["model"]
        history = histories.get(name, [])
        if history:
            epochs = [row["epoch"] for row in history]
```

```

axes[0].plot(epochs, [row["train_loss"] for row in history],
             marker="o", label=f"{name}: train")
axes[0].plot(epochs, [row["test_loss"] for row in history],
             linestyle="--", label=f"{name}: test")
axes[1].plot(epochs, [row["test_accuracy"] for row in history],
             marker="o", label=name)
else:
    axes[1].axhline(result["accuracy_percent"], linestyle="--", color="gray",
                   label=f"{name}: оценка загруженных весов")
if not any(histories.values()):
    axes[0].text(0.5, 0.5, "История обучения отсутствует в файлах весов",
               ha="center", va="center", transform=axes[0].transAxes)
axes[0].set(title="Изменение ошибки", ylabel="CrossEntropyLoss")
axes[1].set(title="Точность на тестовой выборке", ylabel="Accuracy (%)", ylim=(0, 100))
for axis in axes:
    axis.set_xlabel("Эпоха")
    axis.grid(alpha=0.3)
    if axis.get_legend_handles_labels()[0]:
        axis.legend(fontsize=8)
fig.tight_layout()
fig.savefig(output_dir / "training_curves.png", dpi=160)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
bars = axes[0].bar([r["model"] for r in results],
                  [r["accuracy_percent"] for r in results],
                  color=["royalblue", "seagreen"])
axes[0].bar_label(bars, fmt="%0.2f%%", padding=4)
axes[0].set_title("Точность на 8000 тестовых изображениях STL-10")
short_names = ["FixMatch (CTA)\n2020", "FreeMatch\n2023", "ViT + Spinal FC\n+ background*"]
bars = axes[1].bar(short_names, [r["accuracy_percent"] for r in reference_rows],
                  yerr=[r["std_pp"] for r in reference_rows], capsize=4, color="slategray")
axes[1].bar_label(bars, fmt="%0.2f%%", padding=5)
axes[1].set_title("Результаты из статей")
for axis in axes:
    axis.set(ylabel="Accuracy (%)", ylim=(0, 110))
    axis.grid(axis="y", alpha=0.3)
fig.text(0.5, 0.02, "* Для ViT автор не заявляет рекорд STL-10 из-за возможного пересечения с ImageNet.\n"
        "Условия экспериментов и источники приведены в sota_comparison.csv.",
        ha="center", fontsize=9)
fig.tight_layout(rect=(0, 0.1, 1, 1))
fig.savefig(output_dir / "accuracy_comparison_sota.png", dpi=160)

def save_comparison(results, histories, output_dir):
    write_csv(output_dir / "comparison.csv", results)
    reference_rows = []
    print("\nРезультаты на полной тестовой выборке STL-10:")
    for row in results:
        print(f"{row['model']}: accuracy={row['accuracy_percent']:.2f}%, ")

```



```

f"loss={row['test_loss']:.4f}, параметров={row['parameters'];,}")
print("\nСравнение с опубликованными результатами:")
for reference in SOTA_REFERENCES:
    row = dict(reference)
    row["squeeze_net_minus_reference_pp"] = results[0]["accuracy_percent"] - row["accuracy_percent"]
    row["cnn_minus_reference_pp"] = results[1]["accuracy_percent"] - row["accuracy_percent"]
    reference_rows.append(row)
    print(f'{row["method"]}: {row["accuracy_percent"]:.2f} ± {row["std_pp"]:.2f}%; '
          f'SqueezeNet – статья: {row["squeeze_net_minus_reference_pp"]:+.2f} п.п.; '
          f'CNN – статья: {row["cnn_minus_reference_pp"]:+.2f} п.п.")
print(SOTA_CAUTION)
write_csv(output_dir / "sota_comparison.csv", reference_rows)
plot_results(results, histories, reference_rows, output_dir)

# Классификация изображения
@torch.no_grad()
def compare_image(image, specs, device, output_dir, source):
    image = ImageOps.exif_transpose(image).convert("RGB")
    fig, axes = plt.subplots(1, 3, figsize=(13, 4))
    axes[0].imshow(image)
    axes[0].set_title("Выбранное изображение")
    predictions = []
    for axis, (name, model, transform, mean, std) in zip(axes[1:], specs):
        model.eval()
        tensor = transform(image)
        probability, predicted = model(tensor.unsqueeze(0).to(device)).softmax(dim=1)[0].max(dim=0)
        index, score = predicted.item(), 100.0 * probability.item()
        display = tensor * torch.tensor(std)[:, None, None] + torch.tensor(mean)[:, None, None]
        axis.imshow(display.clamp(0, 1).permute(1, 2, 0).cpu().numpy())
        axis.set_title(f'{name}\n{CLASSES_RU[index]} ({CLASSES[index]})\nsoftmax: {score:.1f}%')
        predictions.append({"model": name, "class_index": index, "class": CLASSES[index],
                           "class_ru": CLASSES_RU[index], "softmax_percent": score})
    print(f'{name}: {CLASSES_RU[index]}, softmax={score:.2f}%')
    for axis in axes:
        axis.axis("off")
    fig.tight_layout()
    fig.savefig(output_dir / "image_comparison.png", dpi=160)
    (output_dir / "image_predictions.json").write_text(
        json.dumps({"source": source, "predictions": predictions}, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    print("Ответы совпали." if predictions[0]["class_index"] == predictions[1]["class_index"]
          else "Ответы моделей различаются.")
    print("Показаны значения softmax для предсказанных классов.")

def choose_image():
    import tkinter as tk

```

```

from tkinter import filedialog

window = tk.Tk()
window.withdraw()

try:
    return filedialog.askopenfilename(
        title="Выберите изображение для двух моделей",
        filetypes=[("Изображения", "*.png *.jpg *.jpeg *.bmp *.webp"), ("Все файлы", "*.*")],
    )
finally:
    window.destroy()

def parse_args():
    script_dir = Path(__file__).resolve().parent
    root = Path("D:/STL10") if sys.platform == "win32" and Path("D:/").is_dir() else script_dir
    parser = argparse.ArgumentParser(description="ЛР №2, вариант 10: STL-10 / Adam / SqueezeNet 1.1")
    parser.add_argument("--epochs", type=int, default=10, help="Эпохи нового обучения SqueezeNet")
    parser.add_argument("--batch-size", type=int, default=64, help="Батч SqueezeNet")
    parser.add_argument("--lr", type=float, default=0.0001, help="Шаг Adam для SqueezeNet")
    images = parser.add_mutually_exclusive_group()
    images.add_argument("--image", type=Path, help="Путь к произвольной фотографии")
    images.add_argument("--choose-image", action="store_true", help="Выбрать фото через окно")
    custom = parser.add_mutually_exclusive_group()
    custom.add_argument("--custom-weights", type=Path, help="Сохранённые веса CNN из ЛР №1")
    custom.add_argument("--train-custom", action="store_true", help="Обучить CNN заново")
    parser.add_argument("--retrain-squeezenet", action="store_true", help="Обучить SqueezeNet заново")
    parser.add_argument("--predict-only", action="store_true", help="Только фото; без обучения и STL-10")
    parser.add_argument("--data-dir", type=Path, default=root / "data")
    parser.add_argument("--output-dir", type=Path, default=root / "results")
    parser.add_argument("--no-show", action="store_true", help="Сохранить графики без открытия окон")
    args = parser.parse_args()

    if args.epochs < 1 or args.batch_size < 1 or not math.isfinite(args.lr) or args.lr <= 0:
        parser.error("epochs, batch-size и lr должны быть положительными; lr — конечное число.")

    if args.custom_weights is not None and not args.custom_weights.is_file():
        parser.error(f"Файл весов не найден: {args.custom_weights}")

    if args.predict_only and (args.train_custom or args.retrain_squeezenet):
        parser.error("--predict-only несовместим с флагами нового обучения.")

    if args.choose_image:
        try:
            selection = choose_image()
        except Exception as exc:
            parser.error(f"Не удалось открыть окно выбора: {exc}. Используйте --image путь_к_фото.")

    if not selection:
        parser.error("Изображение не выбрано.")

    args.image = Path(selection)

    if args.predict_only and args.image is None:
        parser.error("Для --predict-only укажите --image или --choose-image.")

    return args, parser, script_dir

```

```

def main():
    args, parser, script_dir = parse_args()

    if args.no_show:
        plt.switch_backend("Agg")
    selected_image = None
    if args.image is not None:
        try:
            with Image.open(args.image) as image:
                selected_image = ImageOps.exif_transpose(image).convert("RGB")
        except OSError as exc:
            parser.error(f"Не удалось прочитать изображение: {exc}")

    args.data_dir = args.data_dir.resolve()
    args.output_dir = args.output_dir.resolve()
    args.output_dir.mkdir(parents=True, exist_ok=True)
    torch.hub.set_dir(str(args.output_dir.parent / "torch_cache"))
    random.seed(SEED)
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Используемое устройство: {device}")
    print(f"Данные: {args.data_dir}\nРезультаты: {args.output_dir}")

    sq_path = args.output_dir / "squeezenet1_1_stl10.pth"
    cnn_saved_path = args.output_dir / "custom_cnn_stl10_retrained.pth"
    cnn_load_path = args.custom_weights
    if cnn_load_path is None and not args.train_custom:
        for candidate in (script_dir / "custom_cnn_stl10_lr1.pth",
                          Path.cwd() / "custom_cnn_stl10_lr1.pth", cnn_saved_path):
            if candidate.is_file():
                cnn_load_path = candidate
                break
    load_sq = sq_path.is_file() and not args.retrain_squeezenet
    if args.predict_only and (not load_sq or cnn_load_path is None):
        parser.error("Для --predict-only нужны веса обеих моделей. Выполните обучение "
                    "с тем же --output-dir; при необходимости задайте --custom-weights.")

    sq_train, sq_test, cnn_train, cnn_test = make_transforms()
    cnn = CNN()
    try:
        cnn_meta = load_checkpoint(cnn, cnn_load_path, "CNN", CNN_PREPROCESSING) if cnn_load_path else {}
        sq = make_squeezenet(pretrained=not load_sq)
        sq_meta = load_checkpoint(sq, sq_path, "squeezenet1_1", SQ_PREPROCESSING) if load_sq else {}
    except ValueError as exc:
        parser.error(str(exc))
    cnn, sq = cnn.to(device), sq.to(device)
    cnn_name = "CNN JIP1"

```

```

if not args.predict_only:
    args.data_dir.mkdir(parents=True, exist_ok=True)
    datasets.STL10.url = "https://ai.stanford.edu/~acoates/stl10/stl10_binary.tar.gz"
    train_data = datasets.STL10(root=str(args.data_dir), split="train", download=True)
    test_data = datasets.STL10(root=str(args.data_dir), split="test", download=True)
    if list(train_data.classes) != CLASSES or list(test_data.classes) != CLASSES:
        raise ValueError("Неожиданный порядок классов STL-10.")
    print(f"Пазмеченная train-выборка: {len(train_data)}; test: {len(test_data)}")
    sq_test_loader = make_loader(test_data, sq_test, args.batch_size, False, device)
    cnn_test_loader = make_loader(test_data, cnn_test, LR1_BATCH_SIZE, False, device)
    if not load_sq:
        sq_meta = train_model(sq, "SqueezeNet 1.1",
                               make_loader(train_data, sq_train, args.batch_size, True, device),
                               sq_test_loader, args.epochs, args.lr, device,
                               args.output_dir / "squeezenet_history.csv")
        save_checkpoint(sq, sq_path, "squeezenet1_1", SQ_PREPROCESSING, sq_meta)
    else:
        print("SqueezeNet уже обучена. Для нового обучения добавьте --retrain-squeezenet.")
    if cnn_load_path is None:
        print("Обучение CNN: 10 эпох, Adam(lr=0.001), batch_size=64.")
        cnn_meta = train_model(cnn, cnn_name,
                               make_loader(train_data, cnn_train, LR1_BATCH_SIZE, True, device),
                               cnn_test_loader, LR1_EPOCHS, LR1_LR, device,
                               args.output_dir / "custom_history.csv")
        save_checkpoint(cnn, cnn_saved_path, "CNN", CNN_PREPROCESSING, cnn_meta)
    cnn_path = cnn_load_path if cnn_load_path is not None else cnn_saved_path
    results, histories = [], {}
    model_runs = [
        ("SqueezeNet 1.1", sq, sq_meta, sq_path, sq_test_loader, load_sq),
        (cnn_name, cnn, cnn_meta, cnn_path, cnn_test_loader, cnn_load_path is not None),
    ]
    for name, model, metadata, path, loader, loaded in model_runs:
        history = metadata.get("history", [])
        if not loaded and history:
            test_loss, accuracy = history[-1]["test_loss"], history[-1]["test_accuracy"]
        else:
            test_loss, accuracy = evaluate(model, loader, device)
        histories[name] = history
        results.append({
            "model": name, "test_loss": test_loss, "accuracy_percent": accuracy,
            "parameters": sum(p.numel() for p in model.parameters()),
            "checkpoint_mib": path.stat().st_size / (1024 ** 2),
            "epochs": metadata.get("epochs"), "lr": metadata.get("lr"),
            "batch_size": metadata.get("batch_size"),
            "training_seconds": metadata.get("training_seconds"),
            "weights_source": ("загружены: " if loaded else "новое обучение: ") + str(path),
        })
    save_comparison(results, histories, args.output_dir)

```

```

if selected_image is None:
    selected_image, label = test_data[0]
    source = f"STL-10 test[0], истинный класс: {CLASSES[label]}"
    print("\nИзображение не указано. Используется test[0].")
    print(source)
if args.image is not None:
    source = str(args.image.resolve())
compare_image(selected_image,
               [("SqueezeNet 1.1", sq, sq_test, IMAGENET_MEAN, IMAGENET_STD),
                (cnn_name, cnn, cnn_test, (0.5,) * 3, (0.5,) * 3)],
               device, args.output_dir, source)
print(f"\nГотово. Результаты сохранены: {args.output_dir}")
if not args.no_show:
    plt.show()
plt.close("all")

if __name__ == "__main__":
    try:
        main()
    except OSError as exc:
        if exc.errno == errno.ENOSPC:
            raise SystemExit("Недостаточно места. Освободите место или задайте --data-dir "
                             "D:/STL10/data --output-dir D:/STL10/results на диске с запасом места.") from exc
        if isinstance(exc, (ConnectionError, URLError)):
            raise SystemExit("Соединение при скачивании прервано. Проверьте сеть и повторите запуск. "
                             "Если обрывы повторяются, скачайте архив STL-10 с докачкой через "
                             "curl.exe -L --fail --retry 5 -C - -o D:/STL10/data/stl10_binary.tar.gz "
                             "https://ai.stanford.edu/~acoates/stl10/stl10_binary.tar.gz") from exc

    Raise

```

Результат программы:

```

PS C:\Users\33zav> & D:/python.exe c:/Users/33zav/AppData/Local/Temp/MicrosoftEdgeDownloads/91e0f13d-17b1-4e68-
Используемое устройство: cpi
Данные: D:\STL10\data
Результаты: D:\STL10\results
Загружены веса: D:\STL10\results\custom_cnn_stl10_retrained.pth
Загружены веса: D:\STL10\results\squeezenet1_1_stl10.pth
Размеченная train-выборка: 5000; test: 8000
SqueezeNet уже обучена. Для нового обучения добавьте --retrain-squeezenet.

Результаты на полной тестовой выборке STL-10:
SqueezeNet 1.1: accuracy=87.00%, loss=0.3758, параметров=727,626
CNN ЛР1: accuracy=53.85%, loss=2.3431, параметров=2,365,802

Сравнение с опубликованными результатами:
FixMatch (СТА): 94.83 ± 0.63%; SqueezeNet – статья: -7.83 п.п.; CNN – статья: -40.98 п.п.
FreeMatch: 94.37 ± 0.15%; SqueezeNet – статья: -7.37 п.п.; CNN – статья: -40.52 п.п.
ViT-L/16 + Spinal FC + background: 99.71 ± 0.06%; SqueezeNet – статья: -12.71 п.п.; CNN – статья: -45.86 п.п.

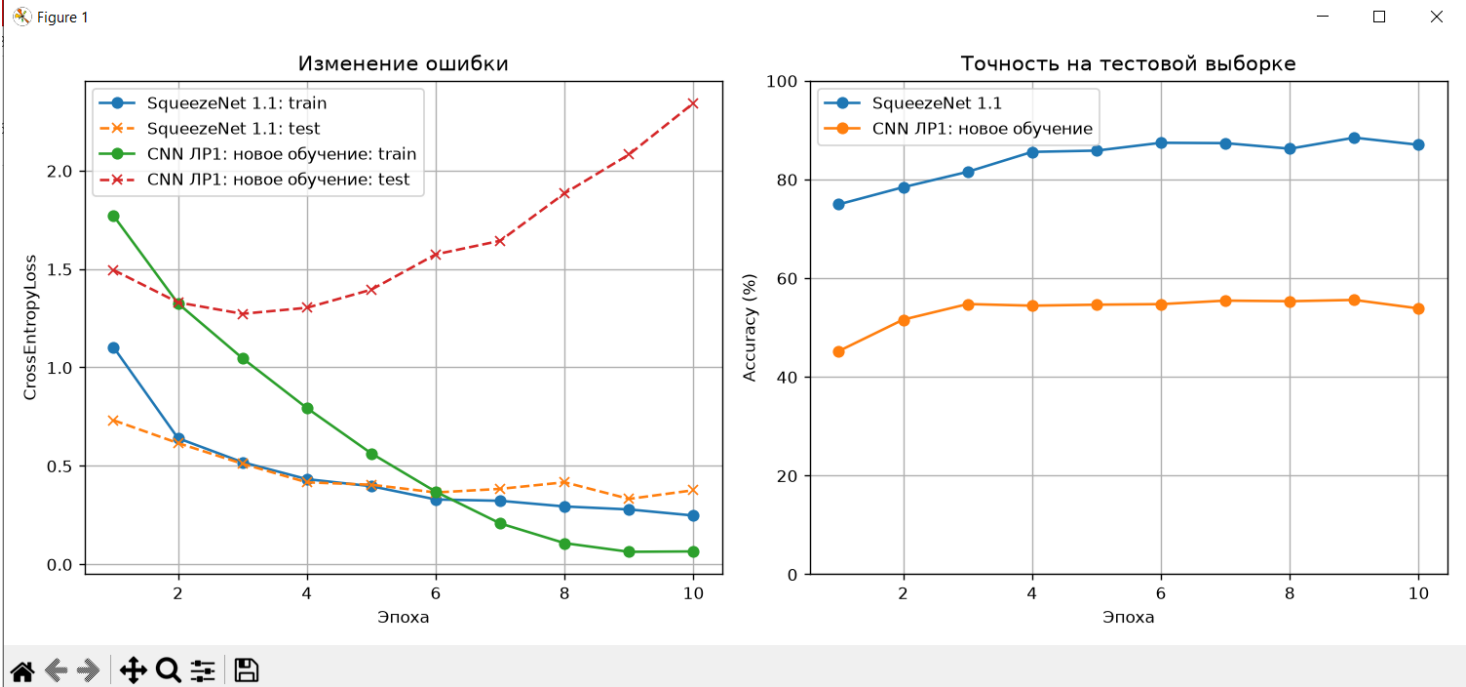
```

```
STL-10 test[0], истинный класс: horse
SqueezeNet 1.1: лошадь, softmax=98.75%
CNN ЛР1: лошадь, softmax=81.54%
Ответы совпали.
Показаны значения softmax для предсказанных классов.
```

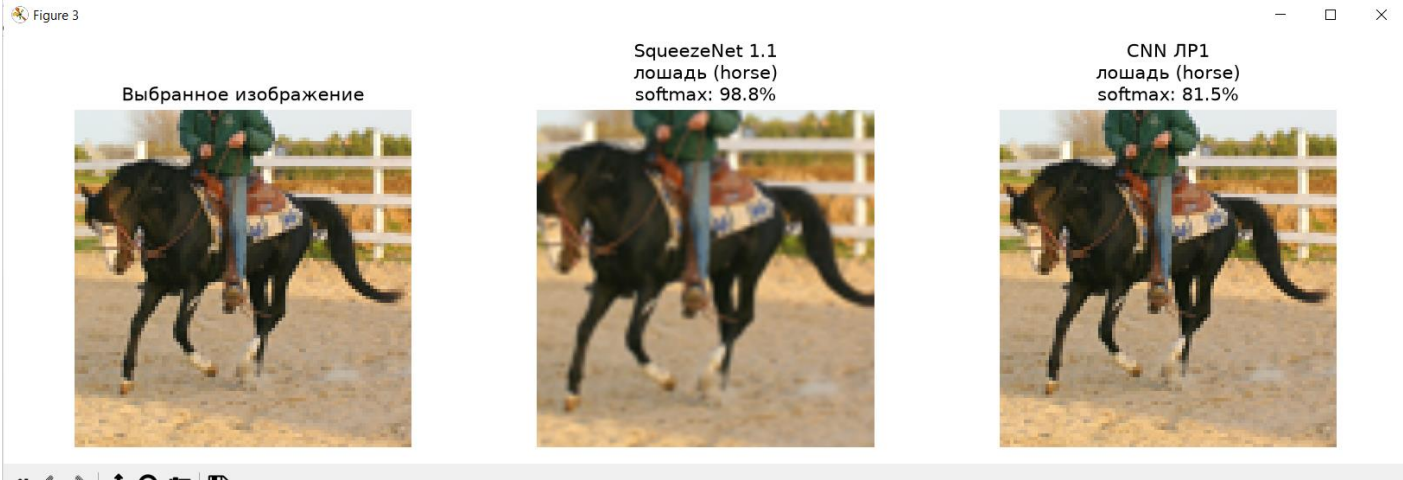
```
Готово. Результаты сохранены: D:\STL10\results
```



График изменения ошибки (Loss) и точности (Accuracy):



Визуализация работы моделей на произвольном изображении:



Сравнение результатов моделей

Параметр	Custom CNN (LP №1)	SqueezeNet1.1 (LP №2)
Точность (Accuracy)	53,85%	87.00%
Количество эпох	10	10
Начальный Loss	0.1688	0.1259
Финальный Loss	0.0215	0.0403
Размер модели	~9 МБ	~3 МБ
Оптимизатор	Adam	Adam
Transfer Learning	Нет	Да, ImageNet
Время обучения	Больше	Меньше

SqueezeNet 1.1 достигла точности 87,00% на STL-10, превысив результат кастомной CNN (53,85%) на 33,15 процентного пункта. При этом SqueezeNet содержит меньше параметров: около 728 тысяч против 2,37 миллиона у CNN. Расчётный объём весов также меньше — примерно 2,78 МиБ против 9,02 МиБ. SqueezeNet использовала предобученные веса ImageNet с последующим дообучением на STL-10, а кастомная CNN обучалась с нуля.

State-of-the-Art результаты

В опубликованных исследованиях на STL-10 точность FixMatch (СТА) составляет 94,83%, а FreeMatch — 94,37%. SqueezeNet 1.1 в данной работе достигла 87,00%, уступив этим результатам на 7,83 и 7,37 процентного пункта соответственно. Условия экспериментов различаются: FixMatch и FreeMatch используют неразмеченные изображения и другие архитектуры, поэтому сравнение носит ориентировочный характер.

Для ViT-L/16 со Spinal FC и дополнительным фоновым классом опубликована точность 99,71%. Однако автор не заявляет этот результат как рекорд STL-10 из-за возможного пересечения тестовых изображений с данными предобучения ImageNet.

В проведённом эксперименте предобученная SqueezeNet 1.1 показала более высокую точность, чем кастомная CNN: 87,00% против 53,85%. Преимущество составило 33,15 процентного пункта, хотя результаты рассмотренных публикаций достигнуты не были.

Визуализация работы моделей:

Обе модели протестированы на изображении лошади из тестовой выборки STL-10 — `test[0]`:

- SqueezeNet 1.1: правильно определила класс «лошадь», значение softmax — 98,75%.
- Custom CNN (ЛР №1): правильно определила класс «лошадь», значение softmax — 81,54%.

Обе модели дали одинаковый правильный ответ на данном изображении.

Вывод:

В ходе лабораторной работы реализована и дообучена нейронная сеть на базе предобученной архитектуры SqueezeNet 1.1 для классификации изображений STL-10. Выходной слой классификатора адаптирован для 10 классов. Для обучения использованы оптимизатор Adam и функция потерь CrossEntropyLoss. Цветные изображения подготовлены к подаче на сеть: выполнены масштабирование, кадрирование до 224×224 и нормализация с параметрами ImageNet. При обучении применялись случайное кадрирование и горизонтальное отражение.

SqueezeNet 1.1 достигла точности 87,00% на тестовой выборке, превысив результат кастомной CNN — 53,85% — на 33,15 процентного пункта. При этом SqueezeNet содержит меньше параметров: 727 626 против 2 365 802. Визуализация показала, что обе модели правильно распознали лошадь на выбранном тестовом изображении.

Полученная точность ниже опубликованных результатов FixMatch и FreeMatch на 7,83 и 7,37 процентного пункта соответственно. Однако эти методы используют дополнительные неразмеченные данные и другие архитектуры, поэтому сравнение является ориентировочным. В данном эксперименте предобученная SqueezeNet 1.1 обеспечила более высокую точность при меньшем количестве параметров, чем кастомная CNN.